

Monitoring & Observability Question Index - All 90 Questions

Q#	Question
Q1	What is observability? Monitoring vs observability - three pillars
Q2	Metrics - counters, gauges, histograms, summaries
Q3	Logs - structured vs unstructured, log levels
Q4	Structured logging - JSON, correlation IDs, context fields
Q5	Distributed tracing - spans, traces, context propagation
Q6	Prometheus architecture - scraping, TSDB, exporters, service discovery
Q7	PromQL basics - selectors, functions, aggregations
Q8	Grafana - dashboards, data sources, panels, variables
Q9	ELK/EFK stack - Elasticsearch, Logstash/Fluent Bit, Kibana
Q10	Loki - labels, LogQL, comparison with Elasticsearch
Q11	Alertmanager - routing, grouping, silencing, inhibition
Q12	CloudWatch - metrics, logs, alarms, dashboards
Q13	Datadog / New Relic / Dynatrace - commercial platforms
Q14	Node Exporter and kube-state-metrics
Q15	Health checks, synthetic monitoring, uptime monitoring
Q16	SLIs, SLOs, SLAs - defining and measuring reliability
Q17	Error budgets - when to stop shipping and fix reliability
Q18	Four golden signals - latency, traffic, errors, saturation
Q19	RED method and USE method
Q20	Metrics vs events vs logs - when to use which
Q21	Metrics naming conventions and label best practices
Q22	On-call and incident management - PagerDuty, OpsGenie
Q23	PromQL deep dive - rate, increase, histogram_quantile

Q24	Prometheus recording rules and alerting rules
Q25	Prometheus service discovery - static, file, Kubernetes, consul
Q26	Prometheus storage - TSDB, WAL, blocks, retention, compaction
Q27	Prometheus federation and remote write
Q28	Thanos vs Cortex vs Mimir vs VictoriaMetrics
Q29	Grafana advanced - provisioning, alerting, annotations
Q30	Grafana alerting vs Alertmanager - when to use which
Q31	Fluent Bit vs Fluentd - architecture, filters, parsers
Q32	Log parsing and enrichment - grok, regex, K8s metadata
Q33	OpenTelemetry - OTLP, collectors, auto-instrumentation
Q34	Jaeger, Zipkin, Grafana Tempo - tracing backends
Q35	Exemplars - linking metrics to traces
Q36	Kubernetes monitoring - Prometheus Operator, ServiceMonitor
Q37	Container and Pod monitoring - cAdvisor, OOM detection
Q38	APM - instrumentation, latency breakdown, dependency maps
Q39	Custom metrics - business metrics, StatsD, OpenTelemetry
Q40	Dashboard design - golden dashboard, anti-patterns
Q41	Alert fatigue - noise reduction, meaningful alerts
Q42	Log-based metrics and alerting
Q43	Infrastructure monitoring - servers, network, databases
Q44	Database monitoring - slow queries, connections, lag
Q45	Network monitoring - packet loss, latency, DNS
Q46	Cost of observability - storage, ingestion, retention
Q47	Monitoring as Code - Terraform, jsonnet, dashboard CI/CD
Q48	Correlation - linking metrics, logs, traces
Q49	Blackbox vs whitebox monitoring

Q50	Service mesh observability - Istio/Linkerd metrics
Q51	Runbooks and automated remediation
Q52	Observability for serverless
Q53	Observability at scale - 1000+ services
Q54	High cardinality - why it kills Prometheus
Q55	Sampling strategies - head, tail, adaptive
Q56	Multi-cluster and multi-cloud observability
Q57	Observability-driven development
Q58	AIOps - anomaly detection, predictive alerting
Q59	Chaos engineering and observability
Q60	Incident management lifecycle
Q61	Postmortem culture - blameless postmortems
Q62	SRE practices - toil, error budgets, reliability
Q63	eBPF-based observability - Hubble, Pixie
Q64	Security and compliance monitoring
Q65	Building an observability platform - buy vs build
Q66	Prometheus OOM - cardinality explosion
Q67	Grafana dashboard loading slow
Q68	Alerts firing but nobody noticed
Q69	Application latency 10x - trace root cause
Q70	Log pipeline dropping messages - backpressure
Q71	CloudWatch costs \$2000/month - optimize
Q72	Elasticsearch cluster red - shards unassigned
Q73	False positive alerts - CPU alarm every deployment
Q74	Service degraded but no alerts - monitoring gap
Q75	Tracing shows 5-second gap - missing span

Q76	Prometheus scrape targets disappearing
Q77	Error budget exhausted - freeze deployments?
Q78	PII in application logs - incident response
Q79	Monitoring the monitoring - watchmen problem
Q80	Prometheus TSDB corruption after disk full
Q81	Loki query timeout on large time range
Q82	Alert storm during deployment - 200 alerts
Q83	Metrics healthy but users report errors
Q84	OpenTelemetry collector OOM - span flood
Q85	Dashboard numbers differ from CLI - aggregation
Q86	Monitoring stack down during incident
Q87	Migrating from Datadog to Prometheus/Grafana
Q88	Log storage costs growing 40% monthly
Q89	Trace shows 99% in one service but metrics fine
Q90	Design observability stack for 50-service platform

SECTION 1 - Junior Core Concepts (Q1-Q22)

Q1 What is observability? Monitoring vs observability - three pillars

The Simple Answer

Monitoring tells you WHEN something is wrong (CPU is at 95%, error rate spiked). Observability tells you WHY something is wrong (which service, which endpoint, which query, which user). Monitoring is about known unknowns - dashboards for metrics you expect to matter. Observability is about unknown unknowns - the ability to ask arbitrary questions about your system without deploying new code.

The Real-World Analogy

Monitoring is like the warning lights on your car dashboard - engine light, oil pressure, temperature. They tell you something is wrong. Observability is like having a full diagnostic computer - it tells you which cylinder is misfiring, at what RPM, under what conditions, and what the exhaust gas composition looks like. Warning lights are necessary but insufficient for diagnosis.

The Three Pillars

Metrics - Numeric measurements over time. CPU utilization at 85%. Request rate of 1,200 req/s. Error rate of 2.3%. Metrics are cheap to store, fast to query, and ideal for alerting and dashboards. But they are pre-aggregated - you lose individual request details.

Logs - Timestamped text records of discrete events. "2025-06-15T10:23:45 ERROR user=alice order_id=12345 payment failed: insufficient funds." Logs are rich in context but expensive to store and slow to search at scale. They capture what happened in detail.

Traces - End-to-end request paths across services. A single user request traced through the API gateway → auth service → order service → payment service → database. Each hop is a span with timing data. Traces reveal where time is spent and which service is the bottleneck.

How They Work Together

A metric alerts you that error rate is 5%. Logs tell you the errors are "payment failed: timeout." Traces show the timeout happens between the payment service and the payment gateway, specifically on retry attempts. Without all three, you are guessing.

The interview-winning statement: "Metrics tell you there is a problem. Logs tell you what the problem is. Traces tell you where the problem is. You need all three for real observability."

Q2 Metrics - counters, gauges, histograms, summaries

The Simple Answer

Metrics are numeric measurements collected over time. There are four fundamental metric types, each serving different purposes.

Metric Types

Counter - Monotonically increasing value. Only goes up (or resets to zero on restart). Examples: total HTTP requests (http_requests_total), total errors (http_errors_total), total bytes sent. Use rate() in PromQL to get the per-second rate of increase.

Gauge - Value that goes up and down. Represents a current state. Examples: current CPU utilization (node_cpu_usage), current memory usage, active connections, queue depth, temperature.

Histogram - Samples observations and counts them in configurable buckets. Records the distribution of values. Examples: request latency distribution - how many requests took 0-100ms, 100-250ms, 250-500ms, 500ms-1s, 1s+. Enables calculating percentiles (p50, p95, p99) using `histogram_quantile()`.

Summary - Similar to histogram but calculates percentiles client-side. Less flexible - cannot aggregate across instances. Histograms are preferred in most cases because they support server-side aggregation.

The practical difference: Use counters for things you count (requests, errors, bytes). Use gauges for things you measure at a point in time (CPU, memory, queue size). Use histograms for things where distribution matters (latency, response size).

Q3 Logs - structured vs unstructured, log levels

The Simple Answer

Logs are timestamped records of events. They capture the richest detail about what happened but are the most expensive to store, index, and search.

Structured vs Unstructured

Unstructured: 2025-06-15 10:23:45 ERROR Payment failed for user alice, order 12345. Human-readable but machine-unfriendly. Parsing requires regex. Each application formats differently.

Structured (JSON): `{"timestamp":"2025-06-15T10:23:45Z","level":"ERROR","message":"Payment failed","user":"alice","order_id":"12345","error":"insufficient_funds"}`. Machine-parseable. Consistent fields. Searchable without regex. Filterable by any field.

Always use structured logging in production. The cost of implementing JSON logging is trivial compared to the hours saved when debugging at 3 AM. Every modern logging library supports structured output.

Log Levels

DEBUG - Detailed diagnostic information. Never enable in production (too verbose). **INFO** - Normal operations. Request processed, service started, config loaded. **WARN** - Something unexpected but recoverable. Retry succeeded, connection pool near limit, deprecated API used. **ERROR** - Something failed. Request failed, operation error, but the service continues. **FATAL** - Unrecoverable error. Service is shutting down.

The production rule: Log at INFO level in production. Enable DEBUG temporarily for specific investigations. Alert on ERROR. Page on FATAL. WARN should trigger investigation, not alerts.

Q4 Structured logging - JSON, correlation IDs, context fields

The Simple Answer

Structured logging outputs key-value pairs (typically JSON) instead of free-form text. This enables machine parsing, filtering, and aggregation without regex.

Correlation IDs

A unique ID (UUID) assigned to each incoming request, passed through every service in the chain via HTTP headers (X-Request-ID). Every log line includes this ID. When debugging an issue, filter logs across all services by the correlation ID to see the complete request flow.

Essential Context Fields

Every log line should include: timestamp, level, service name, correlation ID, and the message. Add context-specific fields: `user_id`, `order_id`, `endpoint`, `response_time`, `status_code`. These fields enable log-based queries like "show all ERROR logs for user alice in the payment service in the last hour."

The practical impact: Without correlation IDs, debugging a request across 10 microservices means manually correlating timestamps across 10 log streams. With correlation IDs, it is a single filter: `correlation_id="abc-123"`.

Q5 Distributed tracing - spans, traces, context propagation

The Simple Answer

A trace represents the entire journey of a request through your system. Each service adds a span - a named, timed operation. Spans are nested to show parent-child relationships. Context propagation passes the trace ID through service boundaries via HTTP headers.

The Analogy - Package Tracking

A trace is like a package tracking number. The package (request) passes through multiple facilities (services). Each facility scans the package and records arrival/departure times (spans). The tracking number (trace ID) links all the scans together. Without tracking, you only know the package left origin and arrived at destination - but not where it spent 3 days stuck in customs (the slow service).

Key Concepts

Trace - The complete end-to-end journey. Has a unique trace ID. **Span** - A single operation within a trace. Has: operation name, start time, duration, service name, status, and key-value tags. **Parent span** - The calling operation. **Child span** - The called operation. **Context propagation** - Passing the trace ID via headers (W3C Trace Context: `traceparent` header, or vendor-specific: `X-B3-TraceId` for Zipkin). Without propagation, spans from different services cannot be linked.

Q6 Prometheus architecture - scraping, TSDB, exporters, service discovery

The Simple Answer

Prometheus is a pull-based monitoring system. It scrapes metrics from HTTP endpoints exposed by applications and exporters, stores them in a time-series database (TSDB), evaluates alerting rules, and makes data available for querying via PromQL.

Pull vs Push Model

Prometheus pulls (scrapes) metrics from targets at configured intervals (default 15 seconds). This means targets must expose an HTTP endpoint (typically `/metrics`) in Prometheus exposition format. The pull model simplifies configuration - Prometheus controls what it monitors - and enables easy detection of target failures (a failed scrape indicates the target is down).

Components

Prometheus server - Scrapes targets, stores data, evaluates rules, serves API. **TSDB** - Built-in time-series database on local disk. Efficient compression. Default 15-day retention. **Exporters** - Translate third-party metrics into Prometheus format. Node Exporter (hardware), `mysqld_exporter` (MySQL), `blackbox_exporter` (probing). **Pushgateway** - For batch jobs that cannot be scraped (short-lived). Job pushes metrics to the gateway; Prometheus scrapes the gateway. **Alertmanager** - Handles alerts: deduplication, grouping, routing, silencing.

Service Discovery

Prometheus must know what to scrape. **Static config** - Hardcoded targets. Simple but does not scale. **File-based SD** - Targets in a file, watched for changes. **Kubernetes SD** - Discovers Pods, Services, Endpoints automatically via the Kubernetes API. The standard for K8s monitoring. **Consul/EC2/Azure/GCE SD** - Cloud and service mesh discovery.

Why Prometheus dominates: Open-source, CNCF graduated, Kubernetes-native, massive ecosystem of exporters, powerful query language (PromQL), and the foundation of the entire cloud-native monitoring ecosystem.

Q7 PromQL basics - selectors, functions, aggregations

The Simple Answer

PromQL (Prometheus Query Language) is a functional query language for selecting and aggregating time-series data. Every Grafana dashboard and every Prometheus alert rule is written in PromQL.

Selectors

Instant vector: `http_requests_total{method="GET",status="200"}` - Select time series matching labels at the current time. **Range vector:** `http_requests_total{method="GET"}[5m]` - Select a range of samples over the last 5 minutes. **Label matchers:** `=` (exact), `!=` (not equal), `=~` (regex match), `!~` (regex not match).

Essential Functions

rate() - Per-second rate of increase of a counter over a range. `rate(http_requests_total[5m])` gives the request rate over the last 5 minutes. THE most important function in PromQL.

increase() - Total increase of a counter over a range. `increase(http_requests_total[1h])` = total requests in the last hour.

sum() - Aggregate across label dimensions. `sum(rate(http_requests_total[5m]))` = total request rate across all instances.

avg(), min(), max() - Statistical aggregations.

by() / without() - Group aggregation. `sum by (method)(rate(http_requests_total[5m]))` = request rate per HTTP method.

The three PromQL patterns that cover 80% of dashboards: `rate()` for request rates and error rates. `histogram_quantile()` for latency percentiles. `sum by()` for grouping by dimensions.

Q8 Grafana - dashboards, data sources, panels, variables

The Simple Answer

Grafana is the visualization layer for observability. It connects to data sources (Prometheus, Loki, Elasticsearch, CloudWatch, Datadog, and 100+ others) and displays data in dashboards with panels (graphs, gauges, tables, heatmaps, stat panels).

Key Features

Dashboards - Collections of panels showing related metrics. Organized by service, team, or use case. **Variables** - Template variables (dropdown filters) for namespace, service, instance. One dashboard serves all services.

Annotations - Mark events on graphs - deployments, incidents, config changes. Correlate metric changes with actions. **Alerting** - Grafana 9+ has unified alerting built in. Define alert rules, contact points, and notification policies directly in Grafana.

Dashboard Best Practices

Start with the overview - RED metrics (Rate, Errors, Duration) for the service. Drill down to specific components. Use consistent time ranges across panels. Include deployment annotations. Add links to logs (Loki) and traces (Tempo/Jaeger) for investigation.

Q9 ELK/EFK stack - Elasticsearch, Logstash/Fluent Bit, Kibana

The Simple Answer

Elasticsearch - Distributed search engine storing and indexing logs. Provides full-text search, aggregation, and analytics. Resource-intensive (each node needs significant RAM for JVM heap and OS page cache).

Logstash - Log processing pipeline. Collects, transforms, and ships logs. Heavy (JVM-based). Being replaced by lighter alternatives.

Fluent Bit - Lightweight log collector written in C. Low memory footprint (a few MB). Kubernetes-native. The standard log collector for container environments. Runs as a DaemonSet, reads container logs from /var/log/containers/, enriches with Kubernetes metadata, and ships to Elasticsearch, Loki, or cloud services.

Kibana - Visualization and exploration UI for Elasticsearch. Dashboards, search, log exploration, and analytics.

ELK vs EFK

EFK replaces Logstash with Fluent Bit (or Fluentd). Logstash is heavier and better for complex transformations. Fluent Bit is lighter and better for log collection and basic processing. Most Kubernetes deployments use EFK.

The Cost Problem

Elasticsearch is expensive to operate - each node needs 16-64 GB RAM, SSDs, and careful index management. This is the primary driver of migration to Loki, which is 10-50x cheaper for log storage by not indexing log content.

Q10 Loki - labels, LogQL, comparison with Elasticsearch

The Simple Answer

Loki is Grafana's log aggregation system designed to be cost-effective and operationally simple. Unlike Elasticsearch which indexes every word in every log line, Loki indexes only metadata labels (like Pod name, namespace, container). Log content is stored compressed and unindexed. Searches are filtered by labels first (fast), then grep through matching log lines (slower but targeted).

LogQL

Loki's query language. Label matching: {namespace="production",app="payment-service"}. Filter: {app="payment"} |= "error". Parse: {app="payment"} | json | status_code >= 500. Aggregate: sum(rate({app="payment"} |= "error" [5m])) by (endpoint).

Loki vs Elasticsearch

Feature	Loki	Elasticsearch
Indexing	Labels only	Full-text
Storage cost	10-50x cheaper	Expensive (full index)
Search speed	Fast on labels, slower on content	Fast on everything
Operations	Simple (single binary)	Complex (cluster management)

Best for	Cloud-native, K8s, cost-sensitive	Full-text search, complex analytics
The decision: Use Loki when log volume is high and you primarily filter by labels (service, namespace, level) before searching content. Use Elasticsearch when you need complex full-text search, aggregations across log content, or when log analytics is a core business function.		

Q11 Alertmanager - routing, grouping, silencing, inhibition

The Simple Answer

Alertmanager receives alerts from Prometheus, deduplicates them, groups related alerts, routes them to the right team, and handles silencing and inhibition.

Key Concepts

Routing - A tree of matchers that routes alerts to different receivers. Critical alerts → PagerDuty. Warning alerts → Slack. Infra alerts → ops-team. App alerts → dev-team.

Grouping - Combines related alerts into a single notification. If 50 Pods have the same error, you get one notification with 50 instances listed, not 50 separate pages.

Silencing - Temporarily mute specific alerts. Used during planned maintenance - "silence all alerts for node-X from 2-4 AM during patching."

Inhibition - Suppress alerts when a higher-level alert is firing. If the entire cluster is down (critical), suppress individual Pod alerts (warning) - they are symptoms, not separate problems.

The alert routing principle: Route by severity AND team. Critical goes to PagerDuty (wakes someone up). Warning goes to Slack (reviewed next business day). Info goes to a dashboard (for trends).

Q12 CloudWatch - metrics, logs, alarms, dashboards

The Simple Answer

CloudWatch is AWS's native monitoring service. It collects metrics from all AWS services automatically, stores logs in log groups, triggers alarms based on thresholds, and visualizes everything in dashboards. It is the default starting point for any AWS-based monitoring. For advanced analysis, use CloudWatch Logs Insights (SQL-like queries) and Container Insights (ECS/EKS dashboards).

Limitations

Limited cross-account visibility (without extra setup). Less powerful querying than PromQL. Dashboard capabilities lag behind Grafana. Custom metrics are expensive at scale (\$0.30 per metric per month). Many teams use CloudWatch for AWS infrastructure metrics and Prometheus/Grafana for application metrics.

Q13 Datadog / New Relic / Dynatrace - commercial platforms

The Simple Answer

Commercial observability platforms provide metrics, logs, traces, APM, and more in a single SaaS product. No infrastructure to manage. Powerful features. Significant cost at scale.

Datadog - Broadest feature set. Infrastructure monitoring, APM, logs, synthetics, security, CI visibility. Dominant in the market. Pricing can be surprising - per-host, per-GB, per-span billing adds up fast.

New Relic - Strong APM heritage. Generous free tier (100 GB/month). Full-stack observability. Good for teams starting with APM.

Dynatrace - AI-powered root cause analysis (Davis AI). Automatic instrumentation. Strong in enterprise environments with complex architectures.

Buy vs Build

Commercial platforms cost \$20-50+ per host per month. At 100 hosts, that is \$24,000-60,000/year. Open-source (Prometheus/Grafana/Loki) costs \$0 in software but requires engineering time to operate. The break-even point depends on your team size and operational expertise.

The honest assessment: Commercial platforms are worth it if your team is small (under 5 DevOps engineers) and you value features over cost. Open-source is worth it if you have the expertise, need customization, or the commercial cost exceeds a dedicated engineer's salary.

Q14 Node Exporter and kube-state-metrics

Node Exporter

Exposes hardware and OS metrics from Linux machines. CPU usage (per core, per mode), memory (free, used, cached, buffers), disk (IOPS, throughput, space), network (bytes sent/received, errors, drops), filesystem, and load average. Runs as a DaemonSet on every node. The foundation of infrastructure monitoring.

kube-state-metrics

Exposes Kubernetes object state as metrics. Pod status (running, pending, failed), Deployment replicas (desired vs available), node conditions, PVC status, Job completion. Does not measure resource usage (that is cAdvisor/metrics-server) - it measures Kubernetes object state.

Together they answer different questions. Node Exporter: "Is the node healthy? How much CPU/memory is available?" kube-state-metrics: "How many Pods are running? Is the Deployment healthy? Are any PVCs stuck?"

Q15 Health checks, synthetic monitoring, uptime monitoring

The Simple Answer

Health checks - Internal endpoints (/health, /ready) that report application status. Consumed by Kubernetes probes and load balancers. Show whether the application is running and can serve traffic.

Synthetic monitoring - Simulated user transactions running on a schedule from external locations. "Can a user log in from New York? Can they complete a purchase from London?" Tests the full stack from the user's perspective.

Uptime monitoring - External checks that verify your service is reachable. Pingdom, UptimeRobot, Blackbox Exporter. Alerts when the service is unreachable from outside your network - catches issues that internal monitoring misses (DNS problems, CDN issues, ISP routing).

Blackbox Exporter

Prometheus's tool for probing endpoints over HTTP, HTTPS, DNS, TCP, and ICMP. Measures response time, SSL certificate expiry, HTTP status codes. Deployed inside the cluster to probe internal services and externally to probe public-facing endpoints.

You need both internal and external monitoring. Internal monitoring shows you the application's perspective. External monitoring shows you the user's perspective. An application can report healthy while being unreachable from the internet due to DNS, CDN, or network issues.

Q16 SLIs, SLOs, SLAs - defining and measuring reliability

The Simple Answer

SLI (Service Level Indicator) - A quantitative measurement of service behavior. Example: the proportion of successful HTTP requests (status < 500) out of total requests. Or: the proportion of requests served in under 200ms.

SLO (Service Level Objective) - The target value for an SLI. Example: "99.9% of requests will succeed" or "95% of requests will complete in under 200ms." Internal commitment - the team agrees to maintain this level.

SLA (Service Level Agreement) - A contract with consequences. Usually less strict than the SLO. Example: "99.5% uptime or you get credits." SLAs are external and legally binding. SLOs are internal and operational.

The Relationship

SLO should be stricter than SLA. If your SLA promises 99.5%, your SLO should be 99.9%. The gap between SLO and SLA is your safety margin - you fix things when SLO is breached, before SLA is violated.

The most common SLI/SLO set: Availability: 99.9% of requests return non-5xx status. Latency: 99% of requests complete in under 500ms. These two cover most web services.

Q17 Error budgets - when to stop shipping and fix reliability

The Simple Answer

An error budget is the inverse of your SLO. If your SLO is 99.9% availability, your error budget is 0.1% - you can afford 0.1% of requests to fail. Over a 30-day month, that is approximately 43 minutes of downtime.

How Error Budgets Drive Decisions

When the error budget is healthy (plenty remaining), prioritize shipping features - the risk of deployment failures is acceptable. When the error budget is nearly exhausted, slow down - focus on reliability improvements, fix flaky deployments, improve testing. When the error budget is completely spent, freeze deployments - only reliability-improving changes are allowed.

Error budgets align incentives. Developers want to ship features. SREs want reliability. Without error budgets, these goals conflict. With error budgets, both teams agree: "We can afford to take risks while the budget is healthy. We must focus on reliability when it is not."

Q18 Four golden signals - latency, traffic, errors, saturation

The Simple Answer

Google's SRE book defines four signals to monitor for every service:

Latency - How long requests take. Monitor p50, p95, p99 percentiles. Distinguish between latency of successful requests and failed requests (errors are often fast - a 500 returned in 1ms should not lower your average latency).

Traffic - How much demand the service is handling. Requests per second for web services. Messages per second for queues. Queries per second for databases.

Errors - The rate of failed requests. HTTP 5xx, application-specific errors, timeout errors. Include both explicit errors (5xx) and implicit errors (200 response with wrong content).

Saturation - How full the service is. CPU utilization approaching 100%, memory approaching limits, disk filling up, connection pool exhaustion. Saturation predicts problems before they cause errors.

Q19 RED method and USE method

RED Method (for services)

Rate - Requests per second. **Errors** - Failed requests per second. **Duration** - Distribution of request latency. RED is the golden signals simplified for request-handling services. Build one RED dashboard per service.

USE Method (for resources)

Utilization - How busy the resource is (CPU at 80%). **Saturation** - How overloaded (queue depth, waiting threads). **Errors** - Error events (disk errors, network drops). USE is for infrastructure resources - CPU, memory, disk, network. Build USE dashboards for nodes and infrastructure.

The practical rule: RED for services (things that handle requests). USE for resources (things services run on). Together they cover both the application and infrastructure layers.

Q20 Metrics vs events vs logs - when to use which

The Simple Answer

Metrics - Pre-aggregated numeric data. Cheap to store, fast to query, ideal for dashboards and alerting. Use for: request rates, error rates, resource utilization, business KPIs. Cannot tell you about individual events.

Logs - Individual event records. Rich context. Expensive to store. Use for: debugging specific requests, audit trails, detailed error information, compliance requirements.

Events - Discrete occurrences with metadata. Deployment happened, config changed, incident started. Use for: annotations on dashboards, change tracking, audit trails. Events often overlap with logs but are more structured and less verbose.

The Cost Hierarchy

Metrics are 100x cheaper than logs per data point. A counter incrementing costs nearly nothing. A log line with 200 bytes costs storage, indexing, and retention. Emit metrics for everything you monitor. Emit logs for things you investigate. Do not log what you should be metricing.

Q21 Metrics naming conventions and label best practices

Prometheus Naming Conventions

Use snake_case. Include the unit as a suffix: http_request_duration_seconds, node_memory_free_bytes, process_cpu_seconds_total. Counters end with _total. Use base units (seconds not milliseconds, bytes not megabytes). Prefix with the application or component name: myapp_orders_processed_total.

Label Best Practices

Labels add dimensions: method, status, endpoint, instance. Keep label cardinality low - labels with many unique values (user_id, request_id, IP address) create millions of time series and kill Prometheus. A label with 1000 unique values × 10 metrics = 10,000 time series per target.

The cardinality rule: Never use unbounded values as labels - user IDs, email addresses, UUIDs, IP addresses, query strings. These create cardinality explosions that OOM Prometheus. See scenario Q66.

Q22 On-call and incident management - PagerDuty, OpsGenie

The Simple Answer

On-call is a rotation where engineers are available 24/7 to respond to production incidents. Tools like PagerDuty and OpsGenie manage schedules, escalation policies, and notification channels.

Escalation Policies

Alert fires → Page the primary on-call (SMS + phone call). If not acknowledged in 5 minutes → page the secondary. If not acknowledged in 10 minutes → page the engineering manager. If the incident is not resolved in 30 minutes → involve the incident commander.

On-Call Best Practices

Rotate weekly. Compensate on-call (time off or pay). Limit pages to actionable alerts - every page should require human action. If a page does not require action, it should be a warning in Slack. Review on-call load monthly - more than 2 pages per shift indicates reliability problems. Run Game Days to practice incident response.

SECTION 2 - Mid-Level Implementation (Q23-Q52)

Q23 PromQL deep dive - rate, increase, histogram_quantile

rate() - The Most Important Function

`rate(http_requests_total[5m])` calculates the per-second rate of increase over 5 minutes. Always use `rate()` on counters, never raw counter values (which only go up). The range `[5m]` should be at least 4x the scrape interval (15s default → 1m minimum, 5m recommended). Too short = noisy. Too long = smoothed.

histogram_quantile() - Percentile Calculation

`histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))` calculates the p99 latency. Requires histogram metrics with `_bucket` suffix. Accuracy depends on bucket boundaries - poorly chosen buckets give inaccurate percentiles. Default buckets (.005, .01, .025, .05, .1, .25, .5, 1, 2.5, 5, 10) work for most web applications.

Common Patterns

Error rate: `sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) * 100`.

Request rate by endpoint: `sum by (handler)(rate(http_requests_total[5m]))`.

Top 5 endpoints by latency: `topk(5, histogram_quantile(0.99, sum by (handler, le)(rate(http_request_duration_seconds_bucket[5m]))))`.

The avg of avg trap: `avg(rate(cpu_usage[5m]))` across instances hides spikes. One instance at 95% and one at 5% averages to 50% - looks fine, but one instance is overloaded. Always check `max()` alongside `avg()` for saturation metrics.

Q24 Prometheus recording rules and alerting rules

Recording Rules

Pre-compute expensive PromQL queries and store the result as new time series. Reduces dashboard load times - the expensive query runs once at evaluation interval, not on every dashboard refresh. Example: `record: job:http_requests:rate5m, expr: sum by (job)(rate(http_requests_total[5m]))`.

Alerting Rules

Define conditions that trigger alerts. `alert: HighErrorRate, expr: rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) > 0.05, for: 5m` (wait 5 minutes before firing to avoid transient spikes), labels: severity: critical, annotations: summary and description with template variables.

The for Duration

The `for` clause prevents alerts from firing on brief spikes. `for: 5m` means the condition must be true for 5 consecutive minutes before the alert fires. Too short = false positives. Too long = late detection. 5 minutes is a good default for most conditions.

Q25 Prometheus service discovery - static, file, Kubernetes, consul

Kubernetes Service Discovery

The most important SD mechanism. Prometheus discovers Pods, Services, Endpoints, and Nodes via the Kubernetes API. Typically configured to scrape Pods with specific annotations: `prometheus.io/scrape: "true"`, `prometheus.io/port: "8080"`, `prometheus.io/path: "/metrics"`. ServiceMonitor CRDs (via Prometheus Operator) provide a cleaner, Kubernetes-native way to define scrape targets.

How Targets Are Discovered

Prometheus watches the Kubernetes API for Pod events. When a Pod with matching annotations/labels is created, Prometheus adds it as a scrape target. When the Pod is deleted, the target is removed. Relabeling rules (`__meta_kubernetes_*` labels) extract metadata for target labels.

Q26 Prometheus storage - TSDB, WAL, blocks, retention, compaction

How TSDB Works

Incoming samples are first written to the WAL (Write-Ahead Log) for crash recovery. Every 2 hours, the WAL is compacted into a block - a self-contained directory of compressed data. Blocks are periodically compacted (merged) to improve query efficiency. Retention is controlled by `--storage.tsdb.retention.time` (default 15 days) and `--storage.tsdb.retention.size`.

Sizing

Each sample takes approximately 1-2 bytes (compressed). With 10,000 time series scraped every 15 seconds: $10,000 \times 4 \text{ samples/min} \times 60 \text{ min} \times 24 \text{ hrs} \times 15 \text{ days} \times 1.5 \text{ bytes} \approx 13 \text{ GB}$. Monitor `prometheus_tsdb_head_series` for current series count and `prometheus_tsdb_storage_blocks_bytes` for disk usage.

The critical rule: Never let Prometheus disk fill to 100%. An unclean shutdown corrupts the TSDB. Use `--storage.tsdb.retention.size` to cap disk usage. Monitor disk free space and alert at 80%.

Q27 Prometheus federation and remote write

Federation

A global Prometheus scrapes aggregated metrics from per-cluster Prometheus instances. Each cluster runs its own Prometheus. The global instance federates (pulls) pre-aggregated metrics using recording rules. Useful for centralized dashboards showing cross-cluster metrics.

Remote Write

Prometheus pushes samples to a remote endpoint in real-time. Used with Thanos, Cortex, Mimir, or VictoriaMetrics for long-term storage. The remote backend provides longer retention, cross-cluster querying, and horizontal scaling that single-instance Prometheus cannot.

Q28 Thanos vs Cortex vs Mimir vs VictoriaMetrics

The Simple Answer

All solve the same problem: Prometheus is single-instance with limited retention. These projects add long-term storage, high availability, and multi-cluster querying.

Feature	Thanos	Mimir	VictoriaMetrics
Architecture	Sidecar + store + query	Distributed microservices	Single binary or cluster
Storage backend	Object store (S3, GCS)	Object store	Custom (on disk or S3)
Complexity	Medium	High (many components)	Low (simplest to operate)
HA Prometheus	Deduplication	Native replication	Deduplication

Grafana Labs backed	Yes	Yes (successor to Cortex)	No (independent project)
Best for	Existing Prometheus, moderate scale	Large scale, multi-tenant	Simplicity, high performance

The practical recommendation: Starting fresh → VictoriaMetrics (simplest). Existing Prometheus + need object storage → Thanos. Large enterprise multi-tenant → Mimir. Cortex is being superseded by Mimir (same team, next generation).

Q29 Grafana advanced - provisioning, alerting, annotations

Dashboard Provisioning

Define dashboards and data sources as YAML/JSON files in version control. Grafana loads them on startup. Changes to provisioned dashboards in the UI are overwritten on restart - preventing UI-based drift. This is monitoring as code.

Annotations

Mark events on graphs - deployments, incidents, config changes. Correlate metric changes with actions. "Latency increased at 10:23 - oh, a deployment happened at 10:22." Annotations can be added manually, via API, or automatically from CI/CD pipelines.

Grafana Unified Alerting

Grafana 9+ has built-in alerting that works across all data sources (not just Prometheus). Define alert rules in Grafana, evaluate them against any data source (Prometheus, Loki, CloudWatch, Elasticsearch), and route notifications through contact points. Can replace Alertmanager for simpler setups.

Q30 Grafana alerting vs Alertmanager - when to use which

Alertmanager

Best when: you use Prometheus exclusively, you need complex routing trees, you need inhibition rules, you have existing Alertmanager configuration, and you manage alerts as code in Prometheus rule files.

Grafana Alerting

Best when: you have multiple data sources (Prometheus + CloudWatch + Elasticsearch), you want alert management in the same UI as dashboards, your team is more comfortable with a visual editor, and you do not need advanced Alertmanager features like inhibition.

They Can Coexist

Grafana can send alerts to Alertmanager as a contact point. Use Prometheus alerting rules for Prometheus-specific alerts (evaluated closer to the data) and Grafana alerting for multi-source or cross-data-source alerts.

Q31 Fluent Bit vs Fluentd

The Simple Answer

Fluent Bit - Lightweight (C, ~450 KB). Low memory (a few MB). Best for log collection from containers and forwarding. Runs as a DaemonSet. The standard for Kubernetes log collection.

Fluentd - Heavier (Ruby, ~40 MB). Rich plugin ecosystem (800+). Best for complex log processing, transformation, and routing. Runs as a centralized aggregator.

The Common Pattern

Fluent Bit on every node (DaemonSet) collects and forwards logs. Fluentd as a centralized aggregator receives, transforms, and routes to multiple destinations. For simple pipelines (collect → ship to Loki/Elasticsearch), Fluent Bit alone is sufficient.

Q32 Log parsing and enrichment - grok, regex, Kubernetes metadata

Kubernetes Metadata Enrichment

Fluent Bit's kubernetes filter automatically adds: pod_name, namespace, container_name, labels, and annotations to every log line from a container. This transforms a raw log line into a searchable, filterable record. Without enrichment, you have text with no context. With enrichment, you can filter by namespace, service, and Pod.

Parsing

JSON parser - If applications output JSON logs, Fluent Bit parses them into structured fields automatically. **Regex parser** - For unstructured logs, define regex patterns to extract fields. **Grok patterns** - Named regex patterns for common formats (Apache, Nginx, syslog). Easier to read and maintain than raw regex.

Q33 OpenTelemetry - OTLP, collectors, auto-instrumentation

The Simple Answer

OpenTelemetry (OTel) is the CNCF project that provides a unified, vendor-neutral way to generate and collect telemetry data (metrics, logs, traces). Instead of instrumenting your application with vendor-specific SDKs (Datadog agent, New Relic agent), you instrument once with OTel and send data to any backend.

Components

SDKs - Language-specific libraries for instrumenting applications. Auto-instrumentation available for Java, Python, Node.js, .NET, Go. **OTLP (OpenTelemetry Protocol)** - The standard protocol for transmitting telemetry. **Collector** - Receives, processes, and exports telemetry. Runs as a sidecar or DaemonSet. Supports: receiving OTLP, Prometheus, Jaeger formats. Processing: filtering, sampling, batching, enriching. Exporting: Prometheus, Jaeger, Zipkin, OTLP, vendor backends.

Why OTel Is the Future

Vendor lock-in is eliminated - switch backends by changing the collector config, not re-instrumenting. All three pillars in one framework. CNCF backing with contributions from every major vendor. Auto-instrumentation means minimal code changes.

Q34 Jaeger, Zipkin, Grafana Tempo - tracing backends

Comparison

Jaeger - CNCF graduated. Established. Rich UI with dependency graphs. Supports Elasticsearch, Cassandra, Kafka as storage. Complex to operate at scale.

Zipkin - Older, simpler. Java-based. Good for small deployments. Less active development.

Grafana Tempo - Newest. Designed for massive scale with object storage (S3, GCS). No indexing - searches by trace ID only (relies on exemplars and logs for discovery). Cheapest to operate. Deep integration with Grafana, Loki, and Prometheus.

The recommendation: For new deployments, Tempo provides the best cost-to-value ratio. Combine with Loki for log-to-trace linking and Prometheus exemplars for metric-to-trace linking. Jaeger for teams needing rich search and dependency analysis.

Q35 Exemplars - linking metrics to traces

The Simple Answer

Exemplars attach trace IDs to specific metric samples. When you see a latency spike on a graph, you can click the exemplar dot and jump directly to the trace that caused it. This bridges metrics and traces - you no longer need to manually correlate timestamps.

How It Works

The application includes a trace ID as an exemplar when recording a metric observation. Prometheus stores the exemplar alongside the sample. Grafana displays exemplars as dots on graphs. Clicking the dot opens the trace in Jaeger or Tempo.

Q36 Kubernetes monitoring - Prometheus Operator, ServiceMonitor

Prometheus Operator

Manages Prometheus instances as Kubernetes Custom Resources. Instead of managing Prometheus configuration files, you define Prometheus CRs, ServiceMonitor CRs (which Pods to scrape), and PrometheusRule CRs (alerting and recording rules). The operator translates these into Prometheus configuration automatically.

kube-prometheus-stack (Helm chart)

The all-in-one monitoring stack: Prometheus Operator, Prometheus instances, Alertmanager, Grafana (with pre-built Kubernetes dashboards), Node Exporter, kube-state-metrics, and default alerting rules. One Helm install gives you a complete Kubernetes monitoring stack. The standard starting point for Kubernetes monitoring.

ServiceMonitor

A CRD that defines which services Prometheus should scrape. Matches services by label selectors. Specifies the port and path to scrape. The Prometheus Operator watches for ServiceMonitor changes and updates Prometheus configuration automatically. This is how teams onboard their services to monitoring - create a ServiceMonitor, and Prometheus starts scraping.

Q37 Container and Pod monitoring - cAdvisor, OOM detection

cAdvisor

Built into kubelet. Collects container-level resource usage metrics: CPU, memory, network, filesystem. Exposed at /metrics/cadvisor on the kubelet. Prometheus scrapes these metrics for container resource monitoring.

OOM Detection

When a container exceeds its memory limit, the kernel OOM killer terminates it. Detection: kube_pod_container_status_last_terminated_reason{reason="OOMKilled"} in kube-state-metrics. container_memory_working_set_bytes approaching the memory limit indicates impending OOM. Alert when usage exceeds 80% of limit.

Q38 APM - instrumentation, latency breakdown

The Simple Answer

APM (Application Performance Monitoring) provides end-to-end visibility into application performance. It traces requests through your code, breaking down time spent in: application logic, database queries, HTTP calls to other services, cache operations, and message queue interactions.

Instrumentation Approaches

Auto-instrumentation - Agent or library hooks into popular frameworks automatically (Spring Boot, Express, Django). Captures HTTP handlers, database drivers, HTTP clients with zero code changes. **Manual instrumentation** - Developer adds custom spans for business-specific operations. More precise but requires code changes.

Q39 Custom metrics - business metrics, StatsD, OpenTelemetry

Business Metrics

Beyond infrastructure metrics, instrument business-relevant metrics: `orders_placed_total`, `revenue_dollars_total`, `user_signups_total`, `cart_abandonment_rate`. These connect observability to business outcomes. When `orders_placed_total` drops, it is an immediate signal something is wrong - even if all infrastructure metrics look healthy.

How to Expose

Prometheus client libraries - Instrument directly in application code. Counter, gauge, histogram in Go, Java, Python, etc. Expose on `/metrics` endpoint. **StatsD** - Push metrics via UDP. Lightweight. Good for applications that cannot expose HTTP endpoints. **OpenTelemetry Metrics** - Part of the OTel SDK. Vendor-neutral. Exported via OTLP to any backend.

Q40 Dashboard design - golden dashboard, anti-patterns

The Golden Dashboard (Every Service Needs This)

Row 1: RED metrics - Request rate, Error rate, Duration (p50, p95, p99). **Row 2:** Saturation - CPU, memory, connection pool utilization. **Row 3:** Dependencies - Downstream service latency and error rates. **Row 4:** Deployment annotations - When did the last deployment happen?

Anti-Patterns

Wall of graphs nobody reads - 50 panels on one dashboard. Nobody looks at it. Limit to 10-15 panels. **Vanity metrics** - Total requests ever is meaningless. Show rate of change. **Average-only latency** - Average hides tail latency. Always show p99 alongside p50. **No annotations** - Without deployment markers, you cannot correlate changes with metric movements. **Not actionable** - Every panel should answer a question. If you cannot describe what action you would take based on a panel, remove it.

Q41 Alert fatigue - noise reduction, meaningful alerts

The Simple Answer

Alert fatigue occurs when too many non-actionable alerts train on-call engineers to ignore pages. When everything alerts, nothing alerts. The result: real incidents are missed because the page is assumed to be another false positive.

Principles

Every alert must be actionable - If the response is "wait and see if it resolves itself," it should not be an alert. Make it a dashboard. **Alert on symptoms, not causes** - Alert on "error rate exceeds 5%" (user impact), not "CPU above 80%" (might be normal). **Use severity levels** - Critical (page immediately, user impact), warning (investigate within business hours), info (dashboard only). **Set appropriate thresholds** - Too sensitive = false positives. Too lenient = missed incidents. Use for: duration to avoid transient spikes.

The metric that matters: Pages per on-call shift. Target: fewer than 2 actionable pages per shift. More than 5 indicates alert quality problems. Track and review monthly.

Q42 Log-based metrics and alerting

The Simple Answer

Extract metrics from log content without a full log indexing pipeline. Fluent Bit, Loki, and CloudWatch can count log patterns and expose them as metrics. Example: count lines matching "ERROR" per minute and alert when the count exceeds a threshold.

Loki Approach

`sum(rate({app="payment"} |= "error" [5m]))` creates a metric from log content. Grafana can alert on this LogQL query directly. No Prometheus metric instrumentation needed in the application - useful for legacy applications that cannot be modified to expose metrics.

Q43 Infrastructure monitoring - servers, network, databases

What to Monitor

Servers: CPU (per mode: user, system, iowait), memory (free, available, cached), disk (usage, IOPS, latency), network (bytes, packets, errors), load average. Source: Node Exporter.

Network: Latency between services, packet loss, DNS resolution time, bandwidth utilization. Source: Blackbox Exporter (probing), SNMP Exporter (network devices).

Databases: Query latency (p50, p99), active connections vs max, replication lag, slow query count, cache hit ratio, deadlock count. Source: database-specific exporters (mysqld_exporter, postgres_exporter, mongodb_exporter).

Q44 Database monitoring - slow queries, connections, replication lag

Key Metrics

Slow queries - Queries exceeding a threshold (100ms for OLTP). Track count per minute and actual query text. Identify and optimize the top 5 slowest queries weekly. **Connection pool** - Active connections vs maximum. Alert at 80% of max. Connection exhaustion causes application failures. **Replication lag** - Seconds behind primary for read replicas. Alert when lag exceeds acceptable staleness (5 seconds for most apps, 0 for financial). **Deadlocks** - Count per minute. Should be near zero. Spikes indicate application-level locking issues. **Cache hit ratio** - For databases with internal caches (PostgreSQL shared_buffers, MySQL buffer pool). Low hit ratio means cache is too small or query patterns are not cacheable.

Q45 Network monitoring - packet loss, latency, DNS

What to Monitor

Latency - Round-trip time between services. Blackbox Exporter with ICMP or TCP probes. Alert when inter-service latency exceeds 10ms (same AZ) or 50ms (cross-AZ). **Packet loss** - Any packet loss in a data center network is abnormal. Even 0.1% loss causes TCP retransmissions and latency spikes. **DNS** - Resolution time and failure rate. Monitor both internal (CoreDNS, kube-dns) and external DNS resolution. **Throughput** - Bandwidth utilization per interface. Alert when approaching link capacity.

Q46 Cost of observability - storage, ingestion, retention

Where Money Goes

Metrics storage - Prometheus TSDB or remote storage. Grows with series count × retention. High cardinality = high cost. **Log storage** - Elasticsearch indices or Loki chunks. Grows with log volume × retention. The largest cost component for most organizations. **Trace storage** - Each trace generates many spans. Without sampling, trace storage can exceed log storage. **Ingestion** - Datadog charges per metric, per GB of logs, per span analyzed. CloudWatch charges per PutMetricData call and per GB ingested.

Cost Reduction

Reduce log volume (drop DEBUG, aggregate repetitive logs). Sample traces (keep 10% of successful traces, 100% of error traces). Reduce metric cardinality (fewer labels, drop unused metrics). Shorten retention (15 days for detailed, 1 year for aggregated). Use cheaper storage tiers (S3 for historical data).

Q47 Monitoring as Code - Terraform, jsonnet, dashboard CI/CD

The Simple Answer

Define dashboards, alert rules, and data sources in version-controlled files. Deploy them through CI/CD. No manual dashboard creation in the UI.

Tools

Grafana Terraform provider - Manage dashboards, data sources, alert rules, folders as Terraform resources. **Jsonnet/Graffonnet** - Generate Grafana dashboard JSON programmatically. Define reusable dashboard components. **Prometheus rules in Git** - Store recording and alerting rules in a Git repository. CI validates syntax (promtool check rules). CD applies to Prometheus.

Why It Matters

Dashboard changes are reviewed in PRs. Accidental dashboard deletion is recoverable (Git history). Consistent dashboards across environments. New services get standard dashboards automatically from templates.

Q48 Correlation - linking metrics, logs, traces

The Simple Answer

Correlation connects the three pillars so you can navigate between them for a single request or incident. You see a metric spike → click to see logs for that time period → click a log line's trace ID → see the distributed trace. Without correlation, you manually switch between tools and guess at timestamps.

Implementation

Metric → Trace: Exemplars (trace IDs attached to metric samples). Click exemplar dot on graph → jump to trace. **Trace → Logs:** Include trace ID in structured log fields. Grafana links from Tempo trace view to Loki logs filtered by trace ID. **Log → Trace:** Parse trace ID from log lines. Grafana detects trace IDs in Loki and creates clickable links. **All three need the same time range and service labels** for seamless navigation.

Q49 Blackbox vs whitebox monitoring

Blackbox (External)

Test from outside the system. "Can a user reach the service?" HTTP probes, TCP probes, DNS checks. Catches: network issues, DNS failures, TLS certificate problems, CDN issues. Does not explain why something is broken - only that it is.

Whitebox (Internal)

Instrument from inside the system. Application metrics, logs, traces. Catches: slow queries, memory leaks, application errors, business logic bugs. Cannot catch external access issues that internal systems do not experience.

You need both. Whitebox monitoring says the service is healthy. Blackbox monitoring confirms users can actually reach it. A service can be internally healthy but unreachable due to DNS, firewall, or CDN issues.

Q50 Service mesh observability

The Simple Answer

Service meshes (Istio, Linkerd) inject a sidecar proxy into every Pod. The proxy sees every request and automatically generates: RED metrics per service pair (which service called which, how fast, how many errors), distributed traces (spans added automatically without application instrumentation), and access logs (full request/response details).

What You Get for Free

Service topology map showing all service-to-service communication. Latency breakdown per service hop. Error rates per service pair. mTLS certificate status. Traffic volume between services. All without modifying application code.

Q51 Runbooks and automated remediation

The Simple Answer

A runbook is a step-by-step procedure for responding to a specific alert. Every alert should link to a runbook. When the on-call engineer gets a page at 3 AM, they should not need to figure out what to do - the runbook tells them.

Automated Remediation

Extend runbooks with automation. Alert fires → trigger a Lambda/script that executes the first diagnostic steps automatically. Auto-restart a crashed service. Auto-scale when capacity is low. Auto-rollback when error rate spikes after deployment. The alert notification includes diagnostic results so the engineer starts with context, not from zero.

Q52 Observability for serverless

Challenges

No persistent infrastructure to run agents on. Cold starts create latency spikes that are not application bugs. Short-lived functions make distributed tracing harder. Vendor-specific monitoring (CloudWatch for Lambda, Cloud Monitoring for Cloud Functions) creates tool fragmentation.

Solutions

AWS Lambda: CloudWatch for metrics (invocations, errors, duration, throttles, concurrent executions). X-Ray or OTel for tracing. Structured JSON logging to CloudWatch Logs. Lambda Insights for enhanced monitoring.

Custom metrics: Use embedded metric format (EMF) to extract metrics from log lines without PutMetricData API calls (cheaper). **Correlation:** Include trace ID and request context in every log line. Use X-Ray or OTel for end-to-end tracing through API Gateway → Lambda → DynamoDB.

SECTION 3 - Senior Architecture & Scale (Q53-Q65)

Q53 Observability at scale - 1000+ services

The Simple Answer

At scale, observability itself becomes a distributed system challenge. Millions of time series, terabytes of logs per day, billions of spans per hour. Single-instance tools break. The architecture needs: horizontally scalable metric storage (Thanos/Mimir/VictoriaMetrics), cost-effective log storage (Loki with object storage), sampled trace storage (Tempo with S3), centralized querying across clusters, and strict governance on what gets collected.

Key Principles

Federated collection - Each cluster runs its own Prometheus, Fluent Bit, OTel collectors. Data is forwarded to centralized storage. **Tenancy** - Multi-team access with namespace-level isolation. Teams see only their services. **Cost attribution** - Track observability cost per team/service. Teams that generate excessive cardinality or log volume pay for it (showback). **Standards** - Enforce metric naming conventions, required labels, log format, and trace context propagation across all teams.

Q54 High cardinality - why it kills Prometheus

The Simple Answer

Cardinality is the total number of unique time series. Each unique combination of metric name + label values creates a separate time series. A metric with labels method (5 values) × status (10 values) × endpoint (100 values) = 5,000 time series. Add instance (50 values) = 250,000 series. Add user_id (100,000 values) = 2.5 billion series. Prometheus OOMs.

Why It Kills Prometheus

Prometheus stores every active time series in memory (the 'head block'). Each series costs approximately 3-4 KB of RAM. 1 million series = 3-4 GB RAM just for the head block. 10 million series = 30-40 GB. Labels with unbounded values (user_id, request_id, IP address, query string) create cardinality explosions that exhaust memory.

How to Handle It

Prevent: Never use unbounded values as labels. Review new metrics for cardinality impact before deployment. Use linting tools (prom-label-enforcer). **Detect:** Monitor prometheus_tsdb_head_series. Alert when it grows unexpectedly. Use tsdb_analyze to find high-cardinality metrics. **Fix:** Remove the offending label. If the data is needed, move it to logs or traces where cardinality is not a problem.

Q55 Sampling strategies - head, tail, adaptive

The Simple Answer

At scale, storing every trace is cost-prohibitive and unnecessary. Sampling reduces trace volume while preserving visibility into problems.

Head-based sampling - Decision made at the start of the trace. "Keep 10% of traces." Simple. Uniform. You miss rare events if they fall in the 90% that is dropped.

Tail-based sampling - Decision made after the trace completes. "Keep all traces with errors or latency > 500ms. Sample 10% of healthy traces." Requires buffering complete traces before deciding. More complex but catches all interesting traces.

Adaptive sampling - Adjusts the sampling rate dynamically based on traffic volume. High traffic = lower sample rate. Low traffic = higher rate (or keep everything). Maintains a consistent cost regardless of traffic patterns.

The recommended approach: Tail-based sampling in the OTel Collector. Keep 100% of error traces and slow traces. Sample 5-10% of healthy traces. This captures every problem while controlling costs.

Q56 Multi-cluster and multi-cloud observability

The Architecture

Each cluster/cloud runs local collectors (Prometheus, Fluent Bit, OTel). Data is forwarded to centralized storage using remote write (metrics), log forwarders (logs), and OTLP export (traces). A single Grafana instance queries across all backends. Unified dashboards show services regardless of where they run.

Challenges

Different label schemes across clusters. Clock synchronization for trace correlation. Network latency affecting remote write. Cost of cross-cloud data transfer. Different authentication mechanisms per cloud.

Q57 Observability-driven development

The Simple Answer

Instrument your code BEFORE you write the business logic, not after. Define the SLIs and SLOs before launch, not after incidents. Add metrics, log lines, and trace spans as part of the definition of done for every feature. Treat observability as a feature requirement, not an afterthought.

How It Changes Development

The PR template includes: "What metrics does this change add or modify?" and "What dashboard changes are needed?" and "What alerts should fire if this breaks?" If the answers are empty, the PR is not ready for production.

Q58 AIOps - anomaly detection, predictive alerting

The Simple Answer

AIOps applies machine learning to observability data to detect anomalies humans would miss, predict problems before they cause incidents, and correlate events for faster root cause analysis.

Practical Applications

Anomaly detection - Learn normal metric patterns. Alert when behavior deviates from the baseline, without manually setting thresholds. Useful for metrics with seasonal patterns (traffic that varies by time of day).

Predictive alerting - "Disk will be full in 3 hours at current growth rate." Gives time to act before impact. **Event correlation** - Group related alerts into a single incident. "These 15 alerts are all symptoms of the database failover."

Limitations

ML models need training data - they are useless for new services. False positives erode trust - start with low-confidence anomalies as info, not pages. Explainability is hard - "the AI detected an anomaly" is not actionable without context.

Q59 Chaos engineering and observability

The Simple Answer

Chaos engineering intentionally injects failures (kill a service, add network latency, simulate disk full) to verify that your monitoring detects the problem, alerts fire correctly, runbooks are accurate, and the system recovers.

Game Days

Scheduled chaos experiments with the team. Inject a failure. Verify: did the right alert fire? Did the on-call receive it? Did the runbook help? Did the system recover within the expected time? Game Days are the integration test for your observability stack.

Tools

Chaos Monkey (Netflix), Litmus (CNCF, Kubernetes-native), Gremlin (commercial), AWS Fault Injection Simulator. All inject controlled failures and verify system behavior.

Q60 Incident management lifecycle

The Phases

Detection - Alert fires or user reports issue. Time to detection is a key metric. **Triage** - Assess severity and impact. Is it affecting users? How many? Which services? **Mitigation** - Stop the bleeding. Rollback, failover, feature flag disable, scale up. Mitigation is not root cause fix - it is making users whole quickly. **Root cause investigation** - After mitigation, find why it happened. Review traces, logs, metrics around the incident timeline. **Resolution** - Fix the root cause permanently. Deploy the fix. **Postmortem** - Document what happened, why, and what we will do to prevent recurrence.

Q61 Postmortem culture - blameless postmortems

The Simple Answer

A blameless postmortem focuses on system and process failures, not individual mistakes. "The deployment process allowed an untested change to reach production" not "John deployed a bad config." Blame creates fear. Fear creates hiding. Hiding creates repeat incidents.

Structure

Timeline of events. Impact assessment (users affected, duration, revenue loss). Root cause analysis (5 Whys). Action items (specific, assigned, tracked). What went well (response time, communication, tools that helped). What can be improved.

The most important part of a postmortem is the action items. If postmortems do not result in tracked improvements, they are just paperwork. Review action item completion monthly.

Q62 SRE practices - toil, error budgets, reliability reviews

Toil

Toil is manual, repetitive, automatable work that scales linearly with service growth. Restarting a service manually, responding to the same alert repeatedly, manually provisioning infrastructure. The SRE target: spend less than 50% of time on toil. Track toil as a metric. Prioritize automation for the highest-toil tasks.

Error Budgets in Practice

Monthly review: how much error budget remains? If healthy, ship features. If low, reliability sprint. If exhausted, freeze deployments. Error budget policies must have executive buy-in - otherwise, feature teams ignore the freeze.

Reliability Reviews

For every new service or major change: review architecture for single points of failure, verify monitoring coverage, test alerting, validate runbooks, and confirm rollback procedures. Before the service is in production, not after the first incident.

Q63 eBPF-based observability - Hubble, Pixie

The Simple Answer

eBPF (extended Berkeley Packet Filter) runs sandboxed programs in the Linux kernel. For observability, this means capturing network traffic, system calls, and application behavior without modifying application code or adding sidecars.

Cilium Hubble - Network observability for Kubernetes. Visualizes Pod-to-Pod traffic, DNS queries, HTTP requests, and TCP connections. Built on Cilium's eBPF data plane. No sidecars or application changes.

Pixie - Full application observability via eBPF. Captures HTTP requests, database queries, and application profiles without instrumentation. Stores data locally in the cluster (no external storage needed). Good for ad-hoc debugging.

Why eBPF is the future: No sidecar overhead (Istio proxy uses 50-100 MB per Pod). No application instrumentation needed. Kernel-level visibility covers all languages and frameworks. The trade-off: Linux-only, requires kernel 4.14+, and the tooling is still maturing.

Q64 Security and compliance monitoring

SIEM Integration

Forward security-relevant logs (authentication, authorization, API access, configuration changes) to a SIEM (Splunk, Elastic Security, Chronicle, Sentinel). Correlate events across systems. Detect patterns: brute force attempts, privilege escalation, data exfiltration.

Audit Logging

Log every privileged action: who accessed what, when, from where. Kubernetes audit logs. Cloud provider audit trails (CloudTrail, Cloud Audit Logs). Application-level audit logs. Tamper-proof storage (immutable S3 bucket, separate account).

Drift Detection

Monitor infrastructure configuration for unauthorized changes. AWS Config, GCP Security Command Center, Azure Policy. Alert when configuration deviates from the declared state. Detect: security groups opened, encryption disabled, public access enabled.

Q65 Building an observability platform - buy vs build

The Decision Framework

Buy (Datadog, New Relic, Dynatrace) when: team is small (under 5 DevOps), you want zero operational overhead, you need features immediately, and cost is acceptable.

Build (Prometheus, Grafana, Loki, Tempo) when: you have 5+ engineers who can operate the stack, cost exceeds a dedicated engineer's salary, you need deep customization, or you want to avoid vendor lock-in.

Hybrid: Many organizations use: Prometheus + Grafana for metrics (build), commercial APM for tracing (buy), Loki for logs (build), PagerDuty for alerting (buy). Pick the best tool for each pillar.

SECTION 4 - Tricky Scenarios (Q66-Q90)

Q66 Prometheus OOM - cardinality explosion

Step-by-Step

Step 1 - Identify the cardinality spike. Check `prometheus_tsdb_head_series` over time. A sudden jump indicates new high-cardinality metrics. If Prometheus is down, check logs for "out of memory" messages.

Step 2 - Find the offending metric. Use the Prometheus TSDB admin API: `curl -s localhost:9090/api/v1/status/tsdb | jq '.data.headStats'` shows top series by label. Or use `promtool tsdb analyze` on the data directory.

Step 3 - Identify the label causing explosion. Usually a label with unbounded values (`user_id`, `request_id`, `session_id`, full URL path). A single metric with 100,000 unique label values creates 100,000 time series.

Step 4 - Fix immediately. Add a `metric_relabel_configs` rule in Prometheus config to drop the metric or the offending label. Reload Prometheus config: `kill -HUP $(pidof prometheus)`. The old series will expire after the staleness period (5 minutes).

Step 5 - Fix at source. Remove the unbounded label from the application's instrumentation. Move the high-cardinality data to logs or traces where it belongs.

Step 6 - Prevent. Set up a `prom-label-enforcer` or admission webhook that rejects metrics with known-bad label names. Monitor `prometheus_tsdb_head_series` with an alert for sudden increases (50% jump in an hour). Review new ServiceMonitors for cardinality impact during code review.

Q67 Grafana dashboard loading slow

Step-by-Step

Step 1 - Check the time range. A dashboard querying 30 days of data at 15-second resolution is millions of data points per panel. Reduce to 24 hours or use recording rules for longer ranges.

Step 2 - Check query complexity. Open browser developer tools → Network tab. Identify which panel queries take longest. Common culprits: `rate()` over large ranges, `histogram_quantile()` across many series, regex label matchers that match thousands of series.

Step 3 - Use recording rules. Pre-compute expensive queries. The dashboard queries the pre-computed metric (instant, cheap) instead of the raw query (expensive).

Step 4 - Reduce panel count. More than 15 panels on one dashboard means too many concurrent queries. Split into multiple dashboards with links between them.

Step 5 - Check Prometheus performance. If all dashboards are slow, Prometheus itself may be overloaded. Check CPU, memory, and query latency. Consider scaling horizontally with Thanos/Mimir.

Q68 Alerts firing but nobody noticed

Step-by-Step

Step 1 - Check Alertmanager routing. Was the alert routed to the correct receiver? Common mistake: a new team's alerts fall through to the default receiver (which nobody monitors).

Step 2 - Check the receiver. Is the Slack webhook valid? Is the PagerDuty API key correct? Did the email go to spam? Check Alertmanager logs for delivery errors.

Step 3 - Check silences and inhibitions. Was a silence active that suppressed the alert? Was an inhibition rule triggered by another alert?

Step 4 - Fix routing. Ensure every team's alerts route to team-specific receivers. Add a catch-all receiver that alerts the platform team for unrouted alerts. Test alert routing quarterly with test alerts.

Step 5 - Implement alert verification. Send a test alert daily to verify the end-to-end path works (Prometheus → Alertmanager → PagerDuty → phone call). If the test alert does not arrive, the monitoring system itself is broken.

Q69 Application latency 10x - trace root cause

Step-by-Step

Step 1 - Identify affected endpoints. Check RED dashboard: which endpoints have increased latency? Is it all traffic or specific operations?

Step 2 - Get a sample trace. Find a trace from the affected time period with high latency. In Jaeger/Tempo, search for traces with duration > threshold.

Step 3 - Analyze the waterfall. The trace shows time spent in each span. Identify the span consuming the most time. Common culprits: a database query span taking 5 seconds (slow query), an HTTP call to another service timing out, a retry loop (same downstream call repeated 3 times).

Step 4 - Drill into the slow span. Check span tags for error details. If it is a database query, check the query text and explain plan. If it is an HTTP call, check the target service's RED metrics.

Step 5 - Fix the root cause. Slow query → add index, optimize query. Downstream timeout → fix the downstream service or add circuit breaker. Retry storm → implement exponential backoff.

Q70 Log pipeline dropping messages - backpressure

Step-by-Step

Step 1 - Confirm drops. Fluent Bit metrics: `fluentbit_output_dropped_records_total` or `fluentbit_output_retries_total`. Check Fluent Bit logs for "backpressure" or "chunk limit reached" messages.

Step 2 - Identify the bottleneck. Is the output (Elasticsearch, Loki) unable to keep up? Check output latency and error rate. Is the output service overloaded or down?

Step 3 - Increase buffers. Increase Fluent Bit memory buffer limits (`Mem_Buf_Limit`). Enable filesystem buffering (`storage.type filesystem`) to spill to disk when memory is full.

Step 4 - Fix the destination. If Elasticsearch is slow, check cluster health, add nodes, or reduce index complexity. If Loki is rejecting, check ingestion limits.

Step 5 - Reduce log volume at source. Drop DEBUG logs in the Fluent Bit pipeline (grep filter). Throttle high-volume sources. Sample repetitive log patterns.

Q71 CloudWatch costs \$2000/month - optimize

Step-by-Step

Step 1 - Check CloudWatch billing breakdown. CloudWatch charges for: log ingestion (per GB), log storage (per GB/month), custom metrics (\$0.30 each), dashboard (\$3 per month each), alarms (\$0.10 each), Logs Insights queries, and API calls.

Step 2 - Optimize log ingestion. Identify which log groups generate the most volume: `aws logs describe-log-groups | sort by storedBytes`. Common culprits: VPC Flow Logs (very verbose), Lambda function logs with excessive DEBUG, ECS container logs with health check spam. Reduce verbosity. Set shorter retention periods (7 days instead of never expire). Exclude high-volume non-critical logs.

Step 3 - Optimize custom metrics. Each custom metric dimension costs \$0.30/month. 1000 custom metrics = \$300/month. Use metric math to derive metrics from existing ones instead of publishing new ones. Use embedded metric format (EMF) instead of `PutMetricData` API calls where possible.

Step 4 - Consider alternatives. Export logs to S3 (much cheaper long-term storage). Use Prometheus + Grafana for custom metrics instead of CloudWatch custom metrics. Use Loki instead of CloudWatch Logs for Kubernetes logs.

Q72 Elasticsearch cluster red - shards unassigned

Step-by-Step

Step 1 - Check cluster health. GET `_cluster/health` shows status (red = primary shards unassigned), number of unassigned shards, and active shards.

Step 2 - Find unassigned shards. GET `_cat/shards?v&h=index,shard,prirep,state,unassigned.reason&s=state`. Shows which index shards are unassigned and why.

Step 3 - Common causes. Disk full - Elasticsearch stops allocating shards when disk usage exceeds 85% (watermark). A node left the cluster - its shards need to be reassigned to remaining nodes. Too many shards - the cluster cannot handle the shard count.

Step 4 - Fix disk. Delete old indices (older than retention period). Reduce replica count temporarily. Increase disk. Clean up old snapshots.

Step 5 - Force shard allocation if needed (for stuck shards after the underlying issue is fixed): POST `_cluster/reroute` with the appropriate `allocate` command.

Step 6 - Prevent. Index lifecycle management (ILM) policies to automatically delete old indices. Disk usage alerting at 70%. Reduce shard count - one shard per 50 GB of data, not one shard per index per day.

Q73 False positive alerts - CPU alarm every deployment

Why It Happens

During deployment, containers restart. CPU spikes briefly during JVM startup, cache warming, or connection pool initialization. The CPU alarm fires on the brief spike, even though it is expected and resolves in 2 minutes.

Step-by-Step Fix

Step 1 - Increase the for duration. Change from `for: 1m` to `for: 5m`. The brief deployment spike resolves before 5 minutes, so the alert never fires.

Step 2 - Exclude deployment windows. Add a deployment annotation to Grafana. Create a deployment-in-progress metric. Inhibit alerts when the metric is active. Use Alertmanager inhibition rules: if alert `deployment_active` is firing, suppress CPU alerts.

Step 3 - Change the metric. Alert on sustained CPU above 90% for 10 minutes, not any spike above 80%. Use `avg_over_time` to smooth spikes.

Step 4 - Use error rate instead of CPU. Alert on user-facing symptoms (error rate > 5%) not infrastructure causes (CPU high). CPU can be high and everything is fine. Error rate above threshold always means users are affected.

Q74 Service degraded but no alerts - monitoring gap

Step-by-Step

Step 1 - What was the symptom? Users reported slow responses. But no latency alert fired.

Step 2 - Why didn't the alert fire? The latency alert checks average latency. Average was 200ms (within threshold). But p99 was 5 seconds - 1% of users experienced terrible performance. The average hid the tail.

Step 3 - Fix. Alert on p99 latency, not average. Add error rate alerts for specific error types (timeout errors, which indicate the tail latency problem). Add synthetic monitoring that tests the actual user flow - it catches issues that average metrics hide.

Step 4 - Gap analysis. Review all services for: Do we alert on percentile latency (not just average)? Do we have error rate alerts? Do we have synthetic monitoring? Do we monitor downstream dependencies? Do we have business metric alerts (orders/minute dropped)?

The lesson: Average hides problems. P99 reveals them. Always alert on percentiles for latency. Always have at least one business-level metric alert per service.

Q75 Tracing shows 5-second gap - missing span

What Happened

A trace shows the API gateway span starting at $t=0$, then a gap of 5 seconds, then the database query span at $t=5$. Total trace duration is 6 seconds. Where did 5 seconds go?

Common Causes

Missing instrumentation - The service between API gateway and database is not instrumented. Its processing time appears as a gap. Fix: add tracing instrumentation to the service.

Async processing - The request was queued (SQS, Kafka, internal queue). Queue wait time is the gap. Fix: instrument the queue producer and consumer as separate spans.

Context propagation broken - The trace ID was not forwarded from one service to the next. The downstream spans exist but are in a separate trace. Fix: verify context propagation headers (traceparent, X-B3-TraceId) are forwarded through all services, load balancers, and message queues.

Q76 Prometheus scrape targets disappearing

Step-by-Step

Step 1 - Check Prometheus targets page. Prometheus UI → Status → Targets. Shows all targets and their status (UP, DOWN, or missing entirely).

Step 2 - If targets are missing (not DOWN, but gone). The service discovery is not finding them. For Kubernetes SD: check if the Pod has the required annotations (prometheus.io/scrape: "true"). Check if the ServiceMonitor selector matches the Service labels. Check if the Prometheus instance's serviceMonitorSelector matches the ServiceMonitor labels.

Step 3 - If targets show DOWN. Prometheus found the target but cannot scrape it. Check: is the Pod running? Is the /metrics endpoint responding (curl from within the cluster)? Is the port correct? Is a NetworkPolicy blocking Prometheus from reaching the Pod?

Step 4 - Check for Pod churn. If Pods are being recreated rapidly (CrashLoopBackOff, frequent deployments), targets appear and disappear quickly. Fix the underlying Pod stability issue.

Q77 Error budget exhausted - freeze deployments?

Step-by-Step

Step 1 - Verify the error budget calculation. Is the SLO correctly defined? Is the measurement accurate? False SLO violations (bad metric, wrong calculation) should not trigger a freeze.

Step 2 - If legitimate, enforce the policy. The error budget policy (agreed upon in advance) dictates the response. Typical policy: freeze feature deployments, allow only reliability improvements and bug fixes.

Step 3 - Identify what consumed the budget. Review incidents from the current period. Which incident consumed the most budget? What was the root cause? What prevented faster detection or recovery?

Step 4 - Invest in reliability. Use the freeze period for: fixing the top reliability issues, improving monitoring (catch problems faster), improving rollback speed (recover faster), and adding chaos tests.

Step 5 - Communicate. The freeze is not punishment - it is the agreed-upon response to protect users. Feature teams understand the trade-off: ship fast most of the time, pause and fix when reliability suffers.

Q78 PII in application logs - incident response

Step-by-Step

Step 1 - Stop the leak. Deploy a code fix that removes PII from log output. Or add a Fluent Bit filter to redact PII patterns (credit card numbers, SSNs, email addresses) before they reach the log store.

Step 2 - Purge existing PII. Delete log entries containing PII from Elasticsearch, Loki, CloudWatch, S3. If logs are in immutable storage, this may require legal review.

Step 3 - Assess compliance impact. GDPR: report to DPA within 72 hours if personal data of EU residents was exposed. CCPA: similar notification requirements. Engage legal and compliance teams.

Step 4 - Root cause. A developer logged an entire request object or user struct. Code review did not catch it. No automated PII detection in the log pipeline.

Step 5 - Prevent. Add automated PII detection to the log pipeline (regex patterns for credit cards, SSNs, emails). Add a code review checklist item: "Does this log statement contain PII?" Use structured logging with explicit field selection - log specific fields, never log entire objects. Add CI checks that scan for common PII logging patterns.

Q79 Monitoring the monitoring - who watches the watchmen?

The Problem

If Prometheus is down, who alerts you? If Alertmanager is down, who sends the alert? If the entire monitoring stack crashes, you are blind.

Solutions

External watchdog. Use a separate, independent monitoring service (UptimeRobot, Pingdom, a second Prometheus instance, or a cloud-native service) to monitor the health of your primary monitoring stack. If Prometheus stops responding, the external service pages you.

Dead man's switch. Prometheus sends a heartbeat alert to a service like Deadman's Snitch or PagerDuty's heartbeat check every 5 minutes. If the heartbeat stops, PagerDuty pages. This catches Prometheus failures, Alertmanager failures, and network issues between them.

Redundant Prometheus. Run two Prometheus instances scraping the same targets. Each runs its own Alertmanager. If one fails, the other continues alerting. Thanos deduplicates the data.

The minimum: A dead man's switch heartbeat via an independent path. Cost: \$0-20/month. Value: knowing when your monitoring is down before an incident makes you discover it the hard way.

Q80 Prometheus TSDB corruption after disk full

Step-by-Step

Step 1 - Free disk space. Delete old data, logs, or other files to get at least 20% free space. Prometheus cannot recover on a full disk.

Step 2 - Check TSDB integrity. `promtool tsdb analyze /path/to/data` checks the database. Look for corrupt blocks.

Step 3 - Remove corrupt blocks. If specific blocks are corrupt, delete them from the data directory (each block is a directory). Prometheus will restart cleanly with the remaining blocks. You lose data for the deleted block's time range.

Step 4 - If TSDB is completely corrupt, delete the entire data directory. Prometheus restarts with empty data. Historical data is lost unless backed up or available in remote storage (Thanos/Mimir).

Step 5 - Prevent. Set `--storage.tsdb.retention.size` to cap TSDB size. Monitor disk usage and alert at 70%. Use remote write to external storage so local data loss is not catastrophic.

Q81 Loki query timeout on large time range

Step-by-Step

Step 1 - Narrow the time range. Query 1 hour instead of 7 days. Loki scans log chunks sequentially - larger ranges = more data to scan.

Step 2 - Add label filters. `{namespace="production",app="payment"} |= "error"` is fast. `{ } |= "error"` scans EVERYTHING - extremely slow. Always filter by labels first, then by content.

Step 3 - Use line filters before parsers. `{app="payment"} |= "error" | json` is faster than `{app="payment"} | json | level="error"`. The line filter (`|=`) drops non-matching lines before the expensive JSON parsing.

Step 4 - Increase query timeout. Loki server config: `limits_config.query_timeout`. But this is a band-aid - optimize queries first.

Step 5 - Add more labels. If a single log stream is too large, add more labels to split it (by level, by component). More labels = more streams = smaller streams = faster queries.

Q82 Alert storm during deployment - 200 alerts in 5 minutes

Step-by-Step

Step 1 - Acknowledge and assess. Are these real alerts or deployment artifacts? Check: did a deployment just happen? Are the alerts resolving after 2-3 minutes?

Step 2 - Silence deployment-related alerts. In Alertmanager, create a silence matching the deployment pattern. This stops the page flood while you investigate.

Step 3 - Root cause. During rolling deployment, old Pods are terminated. Briefly, readiness probes fail, health checks fail, and service endpoints change. Alerts for "endpoint down," "high error rate," and "Pod restarting" all fire simultaneously.

Step 4 - Fix alert configuration. Increase for: duration on deployment-sensitive alerts (5-10 minutes instead of 1). Use deployment-aware inhibition - detect deployment in progress and suppress related alerts. Group related alerts in Alertmanager so you get 1 notification with 200 alerts, not 200 separate pages.

Step 5 - Implement deployment annotations. When the CI/CD pipeline deploys, it creates a Grafana annotation AND sets a `deployment_active` metric. Alertmanager inhibits non-critical alerts when this metric is active.

Q83 Metrics healthy but users report errors

What Happened

All dashboards show green. CPU, memory, error rates, latency - everything looks normal. But user complaints are flooding in: "I can't log in," "my payments are failing."

Step-by-Step

Step 1 - Check from the user's perspective. Run synthetic monitoring tests from external locations. If they fail but internal checks pass, the problem is between users and your service - DNS, CDN, load balancer, or network.

Step 2 - Check specific user paths. The error might affect only specific user flows not covered by your metrics. Login uses a different auth service than the one monitored. Payment processing goes through a third-party gateway not monitored.

Step 3 - Check what is NOT monitored. Common blind spots: third-party dependencies (payment gateways, auth providers, SMS services), CDN/DNS layer, client-side errors (JavaScript failures, mobile app crashes), specific API endpoints without monitoring, async processing (queue workers).

Step 4 - Fix the gap. Add synthetic monitoring for critical user flows. Add error rate monitoring for all API endpoints (not just aggregate). Monitor third-party dependency health. Add real user monitoring (RUM) for client-side visibility.

The lesson: Infrastructure metrics are necessary but insufficient. You must monitor the user's experience directly - synthetic tests, RUM, and business metrics (orders/minute, logins/minute).

Q84 OpenTelemetry collector OOM - span flood

Step-by-Step

Step 1 - Identify the source. Check collector metrics: `otelcol_receiver_accepted_spans` and `otelcol_receiver_refused_spans` by service. One service may be sending millions of spans per minute.

Step 2 - Check if sampling is configured. Without sampling, every request generates traces. A service handling 10,000 req/s with 5 spans each = 50,000 spans/second = 3 million spans/minute. This overwhelms any collector.

Step 3 - Add tail-based sampling. Configure the `tail_sampling` processor in the collector. Keep 100% of error traces. Keep 100% of high-latency traces. Sample 5% of healthy traces. This reduces volume by 90%+ while keeping all interesting traces.

Step 4 - Add resource limits. Set `memory_limiter` processor with `check_interval`, `limit_mib`, and `spike_limit_mib`. The collector drops data instead of OOMing - better to lose some traces than lose the entire collector.

Step 5 - Scale the collector. Run the collector as a Deployment (not DaemonSet) with HPA based on memory or queue depth. Scale horizontally to handle the load.

Q85 Dashboard numbers differ from CLI - aggregation confusion

What Happened

The Grafana dashboard shows average CPU at 45%. The engineer runs `kubectl top nodes` and sees one node at 90%. "The dashboard is lying!"

Why It Happens

The dashboard shows `avg(node_cpu_usage)` across all nodes. 10 nodes: one at 90%, nine at 40%. Average = 45%. The average hides the outlier. `kubectl top` shows per-node values, revealing the problem.

The Fix

Never use only `avg()` for resource metrics. Always show `max()` alongside `avg()`. Or show a heatmap that reveals the distribution. Or use `topk(5, ...)` to show the top 5 values.

avg of rate vs rate of avg - These give different results. `avg(rate(metric[5m]))` first calculates per-instance rates, then averages them. `rate(avg(metric)[5m])` first averages raw values, then calculates the rate. The first is almost always what you want.

Q86 Monitoring stack went down during incident

Step-by-Step

Step 1 - Use the backup monitoring. This is why you have external uptime monitoring (Pingdom/UptimeRobot) and a dead man's switch. The external service confirmed the outage even though internal monitoring was down.

Step 2 - Diagnose without dashboards. SSH to nodes. Use CLI tools: kubectl top, docker stats, top, curl health endpoints manually. Check cloud provider console (CloudWatch, GCP Monitoring).

Step 3 - Fix the monitoring stack. Is Prometheus down? Check Pod status. Is it OOM (cardinality explosion during the incident)? Is the storage full? Restart with increased resources.

Step 4 - Post-incident. Add monitoring of the monitoring stack (dead man's switch, external health checks). Run Prometheus with HA (two instances). Store data in remote storage so local failures do not lose data. Document the manual investigation procedure for when dashboards are unavailable.

Q87 Migrating from Datadog to Prometheus/Grafana

Step-by-Step Strategy

Step 1 - Inventory. List all Datadog integrations, dashboards, monitors (alerts), and APM traces. Map each to an open-source equivalent.

Step 2 - Deploy the stack. kube-prometheus-stack (Prometheus + Grafana + Alertmanager + Node Exporter + kube-state-metrics). Loki for logs. Tempo for traces. All via Helm.

Step 3 - Run in parallel. Keep Datadog running. Deploy Prometheus alongside. Compare metrics between both systems for at least 2 weeks. Verify data accuracy.

Step 4 - Migrate dashboards. Recreate critical dashboards in Grafana. Datadog dashboards cannot be directly converted - the query languages are different. Focus on the top 10 dashboards first.

Step 5 - Migrate alerts. Rewrite Datadog monitors as Prometheus alerting rules. Test each alert by injecting failures. Verify routing through Alertmanager.

Step 6 - Cut over. Disable Datadog integrations service by service. Monitor for gaps. Keep Datadog available (but not collecting) for 30 days as a safety net.

Timeline

2 engineers, 2-3 months. Month 1: deploy stack, parallel run. Month 2: migrate dashboards and alerts. Month 3: cut over, decommission.

Q88 Log storage costs growing 40% monthly

Step-by-Step

Step 1 - Identify the growth source. Which log streams are growing fastest? Check per-namespace, per-service log volume. Often one service generates 80% of log volume.

Step 2 - Audit log levels. Are DEBUG logs enabled in production? One DEBUG line per request at 10,000 req/s = 600,000 log lines per minute. Disable DEBUG. Reduce INFO logs to essentials.

Step 3 - Drop noise. Health check logs (every 10 seconds per Pod × 100 Pods = 600 lines/minute of useless logs). Add a Fluent Bit grep filter to drop health check lines. Drop repetitive "connection established" or "heartbeat" messages.

Step 4 - Reduce retention. Do you need 90 days of detailed logs? Keep 7 days detailed, 30 days aggregated, 365 days for compliance-required logs only.

Step 5 - Sample verbose logs. If a service legitimately generates high-volume logs, sample them - keep 10% of successful request logs, 100% of error logs.

Step 6 - Switch to cheaper storage. Loki with S3/GCS backend is 10-50x cheaper than Elasticsearch for the same volume.

Q89 Trace shows 99% time in one service but metrics fine

What Happened

A trace shows 4.8 seconds in the payment service and 0.2 seconds in everything else. But the payment service dashboard shows p99 latency of 50ms.

Why It Happens

Client-side vs server-side measurement. The trace span measures time from the caller's perspective - including network latency, connection time, and queuing. The service's metrics measure only processing time. A 4.8-second span with 50ms server-side processing means 4.75 seconds was spent in: network latency, TCP connection setup, TLS handshake, load balancer queuing, or waiting for an available connection from the connection pool.

Step-by-Step Fix

Step 1 - Check connection pooling. Is the calling service creating a new connection per request? HTTP connection setup + TLS handshake can take seconds. Fix: use connection pooling.

Step 2 - Check load balancer queuing. If the target has too few instances or a long processing tail, the ALB/NLB queues requests. Check ALB target response time and queue depth.

Step 3 - Check network. Cross-AZ or cross-region calls add latency. Check if the services are in the same AZ.

Q90 Design observability stack for 50-service platform

The Architecture

Metrics: kube-prometheus-stack (Prometheus Operator + Grafana + Alertmanager). Remote write to Thanos or VictoriaMetrics for long-term storage and cross-cluster querying. Recording rules for dashboard performance.

Logs: Fluent Bit DaemonSet → Loki with S3 backend. Structured JSON logging enforced across all services. Retention: 14 days detailed, 90 days aggregated.

Traces: OpenTelemetry auto-instrumentation (Java, Python, Node.js agents). OTel Collector with tail-based sampling (100% errors, 5% healthy). Grafana Tempo with S3 backend. Exemplars linking Prometheus metrics to Tempo traces.

Alerting: Prometheus alerting rules for infrastructure and service-level alerts. Alertmanager routing: critical → PagerDuty, warning → Slack. Grafana alerting for cross-data-source alerts.

Correlation: Trace ID in all structured log fields. Exemplars in Prometheus histograms. Grafana's Explore view links metrics → logs → traces.

SLOs: Define SLIs (availability, latency) per service. Implement error budget tracking with Pyrra or Sloth. Monthly reliability review.

Cost: VictoriaMetrics or Thanos with S3 for cheap metric storage. Loki with S3 (no full-text indexing) for cheap log storage. Tempo with S3 for cheap trace storage. Total: approximately \$500-2000/month for 50 services (versus \$20,000-50,000/month for equivalent Datadog).

Governance: Metric naming conventions enforced via linting. Required labels (team, service, environment) on all metrics. Dashboard templates for consistent service dashboards. On-call rotation with PagerDuty. Quarterly Game Days for incident response practice.

Interview Tips

For Junior Roles

Focus on Q1-Q22. Know the three pillars and how they work together. Understand Prometheus pull model and PromQL basics. Know SLI/SLO/SLA definitions. Be able to explain the four golden signals and RED/USE methods.

For Mid-Level Roles

Q1-Q52. Be ready to write PromQL queries on the spot. Understand OpenTelemetry architecture. Know how to design a golden dashboard. Discuss alert fatigue solutions. Have a real story about using traces to debug a latency issue.

For Senior Roles

All 90 questions. High cardinality, sampling strategies, observability at scale, and SRE practices separate senior from mid-level. Be ready to design a complete observability stack and discuss cost trade-offs between commercial and open-source solutions.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Three pillars	Metrics (what), Logs (why), Traces (where). You need all three.
Prometheus	Pull-based. PromQL. 15-day default retention. Watch cardinality.
PromQL essentials	rate() for counters. histogram_quantile() for percentiles. sum by() for grouping.
Grafana	Visualization layer. Connects to everything. Provisioned dashboards as code.
Loki vs Elasticsearch	Loki: label-indexed, cheap, simple. ES: full-text, expensive, powerful.
OpenTelemetry	Vendor-neutral. One instrumentation, any backend. The future standard.
SLI/SLO/SLA	SLI measures. SLO targets. SLA contracts. SLO should be stricter than SLA.
Error budget	Inverse of SLO. Budget healthy = ship. Budget exhausted = freeze + fix.
Golden signals	Latency, traffic, errors, saturation. Monitor all four per service.
RED/USE	RED for services (Rate, Errors, Duration). USE for resources (Utilization, Saturation, Errors).
Alert fatigue	Every alert must be actionable. Target < 2 pages per on-call shift.
Cardinality	Never unbounded labels (user_id, request_id). Monitor head_series.
Sampling	Tail-based: 100% errors, 5% healthy. Controls cost, keeps interesting traces.
Correlation	Trace ID in logs. Exemplars in metrics. Grafana links all three.
Dashboard design	RED row → saturation row → dependencies row → annotations. Max 15 panels.
Monitoring as Code	Dashboards in Git. Recording rules in Git. Alerting rules in Git. CI/CD deploys.
Commercial vs OSS	Small team → buy (Datadog). Large team → build (Prometheus/Grafana/Loki). Hybrid common.
Postmortems	Blameless. Focus on systems. Action items tracked. Monthly review.
Dead man's switch	Heartbeat alert to external service. Catches monitoring stack failures.